

Efficient Query Rewrite Rule Discovery via Standardized Enumeration and Learning-to-Rank

Yuan Zhang, Yuxing Chen, Yuekun Yu, Jinbin Huang, Rui Mao,
Anqun Pan, Lixiong Zheng, Jianbin Qin



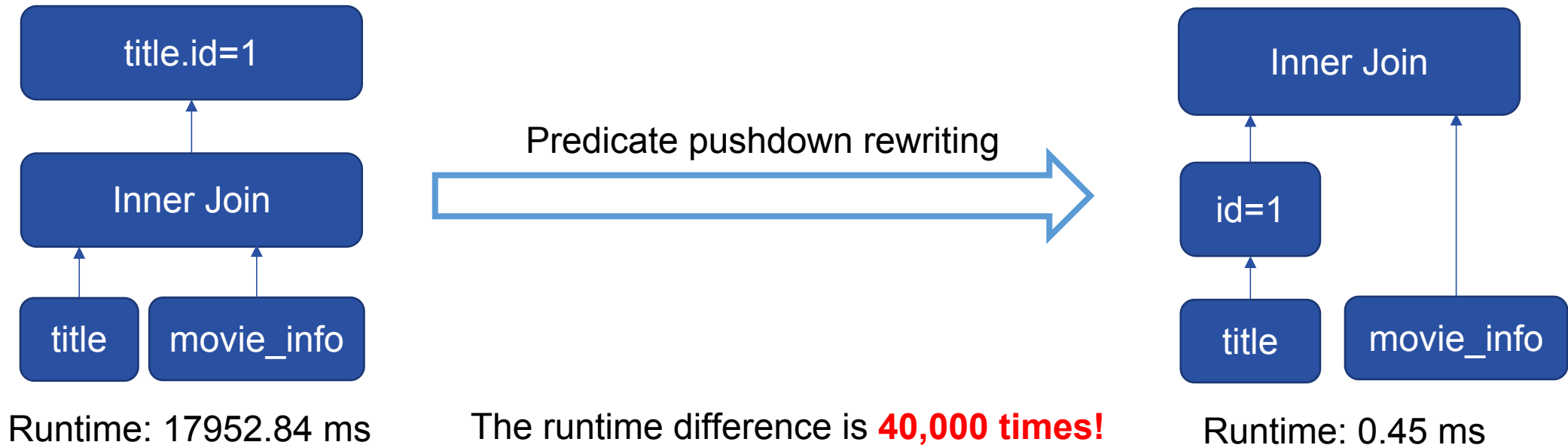
Tencent

Contents

- Background & Motivation
- Limitations of SOTA
- Contribution
- Preliminaries
- Key Idea 1 – Standardized Templates
- Key Idea 2 – RTP Deduplicator
- Key Idea 3 – Learning-to-Rank
- Experiments
- Conclusion

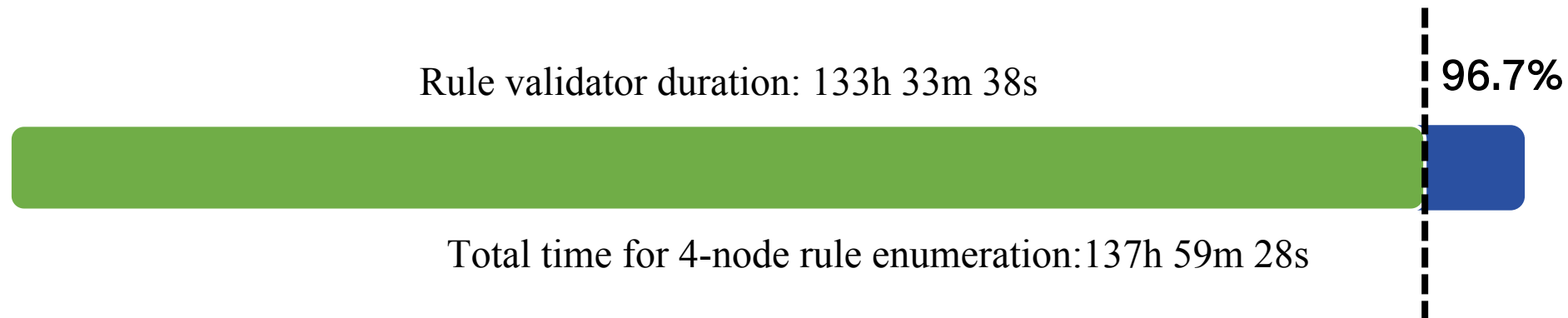
Backgroud & Motivation

- Real-world queries are complex:
 - Most production queries contain **5–8+ nodes** (PostgreSQL, web apps).
 - Auto-generated SQL (ORM, AI tools) produces redundant joins/subqueries.
- Manual rules are **incomplete and unsustainable**.
- Existing optimizers leave many queries unoptimized.



Limitations of SOTA (WeTune[1])

- Brute-force enumeration of plan templates + constraints
 - **4-node rules:** 4,320 CPU hours (~9 days), >90% redundant, >50% trivial
 - **5-node rules:** ~6 months on 64-CPU machine
 - **6+ node rules:** Infeasible (decades)
- Cannot handle real workloads (most queries >4 nodes)
- No effective prioritization → rule base clogged with **low-value rules**

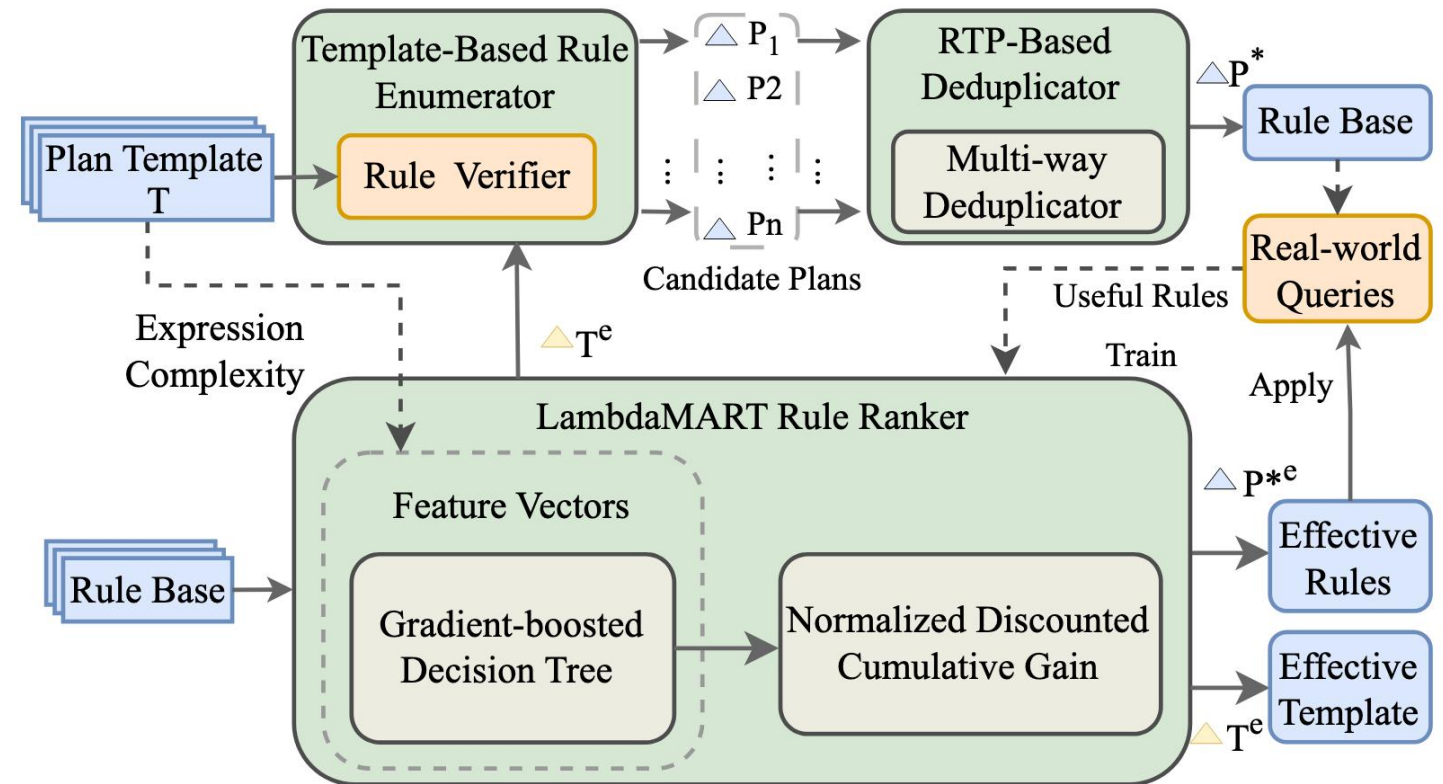


Contribution

- Query rewriting is essential for database performance, yet automated rule discovery remains challenging due to exponential search spaces, massive redundancy (**>90%**), and poor scalability beyond 4-node plans.
- SLER (Standardized Enumeration + Learning-to-Rank) is the first scalable system that discovers >1 million empirically validated rewrite rules — **the largest rule library to date.**
- Supports complex templates up to **channel-level depth** (previously infeasible).

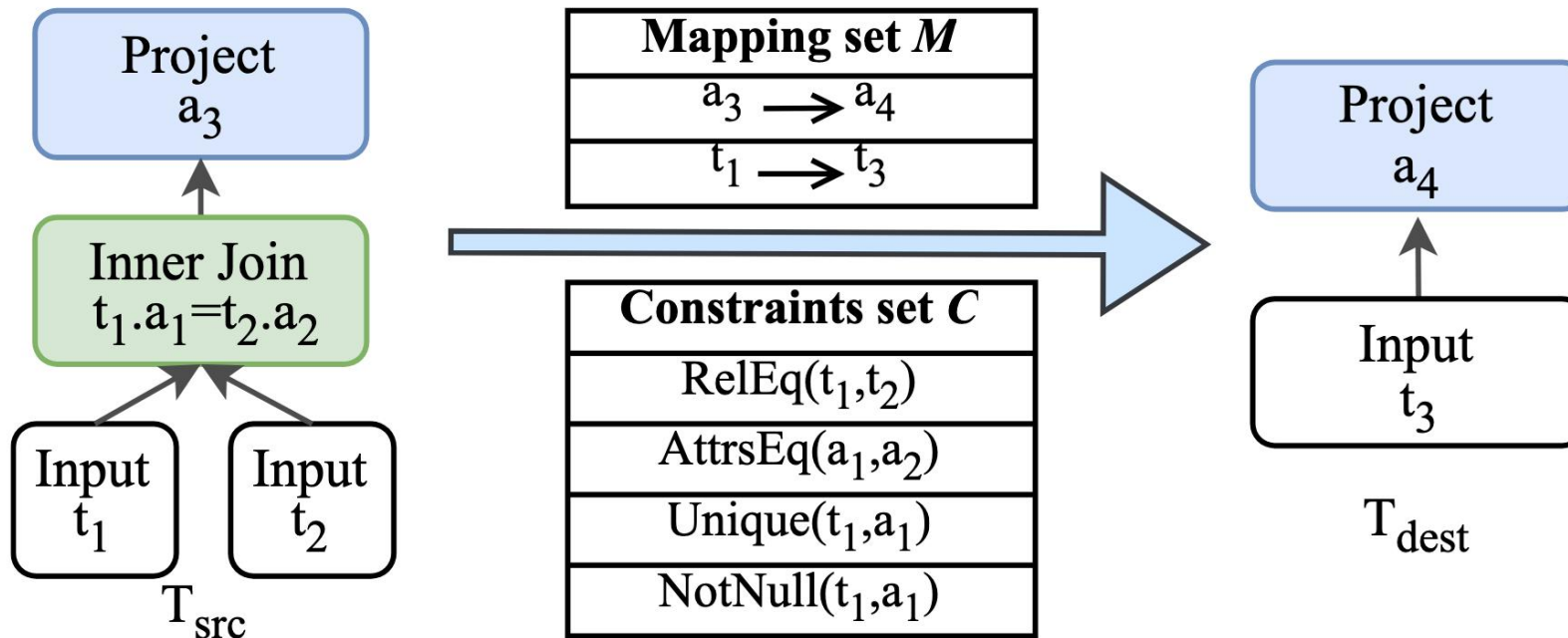
SLER Overview

- **Standardized Template Enumerator** – drastically reduces search space
- **RTP-based Deduplicator** – eliminates redundancy at template-pair level
- **LambdaMART Rule Ranker** – learning-to-rank for high-value rules + pre-filtering



Key Idea 1 – Standardized Templates

- **Rule definition:** A rule $R(T_{src}, T_{dest}, M, C)$ consists of a source template, a target template, rewrite mappings, and constraints.
- **Mining Process:** 1) Select source (T_{src}) and target (T_{dest}) templates. 2) Enumerate rewriting mappings (M). 3) Enumerate the loosest constraint set (C) that validates the rule.



Preliminaries – Query Plan Operators

- A query plan template T is a directed tree where each node is a relational algebra operator.
- Supported Operators (same as WeTune)

Operator	Semantics
Input(R)	Input relation R (table or subquery)
Project(a)	Project attributes in list a
Filter(p, a)	Keep tuples satisfying predicate p on attributes a
InnerJoin($R_l.a_l, R_r.a_r$)	Inner join on $R_l.a_l = R_r.a_r$
LeftJoin / RightJoin	Left / Right outer join (symmetric)
InSub($R_l.a_l, R_r.a_r$)	Semi-join: keep rows from R_l where value exists in R_r
Distinct(a)	Project a and remove duplicates

Constraints

- C is minimal (most relaxed possible): removing any single constraint breaks equivalence
- Constraint Types (used in C)

Constraint	Meaning
$\text{RelEq}(R_1, R_2)$	Relations R_1 and R_2 contain identical tuples
$\text{AttrEq}(a_1, a_2)$	Attribute lists are identical
$\text{PredEq}(p_1, p_2)$	Predicates are logically equivalent
$\text{SubAttr}(a_1, a_2)$	Every attribute in a_1 also appears in a_2
$\text{RefAttr}(R_1.a_1, R_2.a_2)$	All values in $R_1.a_1$ exist in $R_2.a_2$
$\text{Unique}(R, a)$	Values in R on attribute a are unique
$\text{NotNull}(R, a)$	No NULL values in R.a

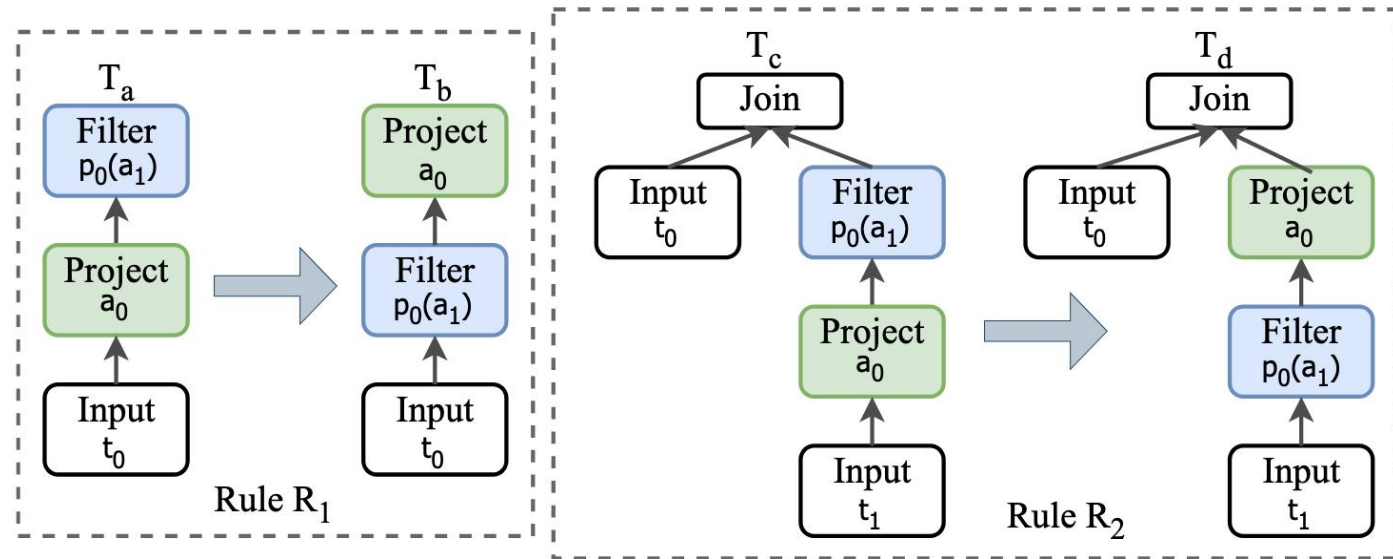
Key Idea 1 – Standardized Templates

- **Observation:** Many rules differ only in trivial operator ordering (Project/Filter/Distinct swaps) or redundant operators.
- **Standardized Rule :** Empty constraint set + only two transformations:

- Swap two adjacent operators
- Remove one operator

- **Priority order :** Project > Distinct > Filter

- **Property:** Any non-standardized rule is provably redundant if a standardized rule exists.



Key Idea 2 – RTP Deduplicator

- **Traditional deduplication:** $O(n^2)$ nested loops $\rightarrow$ >20 days for 4-node rules.
- **RTP Algorithm**
 - Per template pair (T_{src} , T_{dest})
 - Incremental redundancy check during enumeration
 - Uses optimizer to test whether adding a rule changes optimization result
- Multi-way subsumption + subset partitioning (Compared to R_3 , both R_1 and R_2 are redundant rules.)
- Guaranteed correctness (proven in paper).

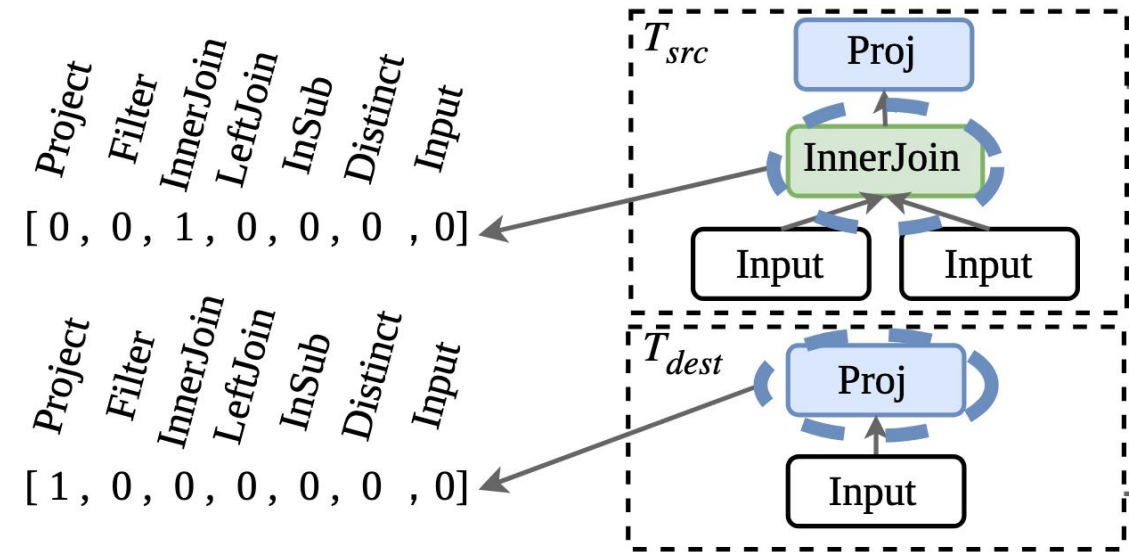
Rule	Source Plan Template	Destination Plan Template	Extra Constraints
R_1	$\text{Proj}_{t_0.a_0}(\text{IJoin}_{t_0.a_1, t_0.a_0}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_0); \text{Unique}(t_0, a_1)$
R_2	$\text{Proj}_{t_0.a_1}(\text{IJoin}_{t_0.a_0, t_0.a_1}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_1); \text{Unique}(t_0, a_1)$
R_3	$\text{Proj}_{t_0.a_0}(\text{IJoin}_{t_0.a_0, t_0.a_1}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_0); \text{NotNull}(t_0, a_1); \text{Unique}(t_0, a_1)$

Key Idea 3 – Learning-to-Rank

- **Problem:** Proof length (Z3 assertions) is unreliable (NP=coNP open problem).
- **Solution:** Train LambdaMART (gradient-boosted trees) on real query workload (>11k SQL queries).

- **Features:**

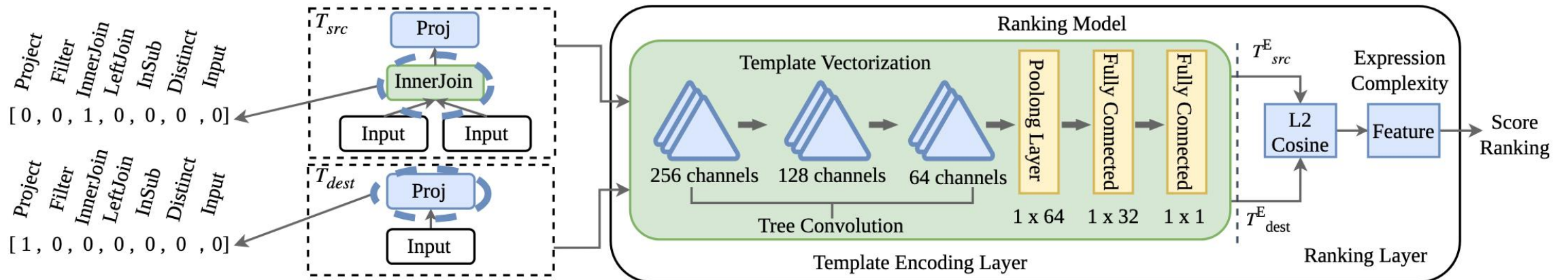
- Tree distance & structural similarity
- Expression complexity
- Operator types



- Encoding each node of the tree using one-Hot encoding

Key Idea 3 – Learning-to-Rank

- The Figure below illustrates the architecture of rule effectiveness ranking.
 - The vectorized tree is passed through three layers of tree convolution.
 - After the last layer of tree convolution, the tree structure is flattened into a single vector.
 - The source template and target template tree(T_{src}, T_{dest}) output a fixed vector (V_{src}, V_{dest}).



LambdaMART – Quantitative Features

We transformed the operators into expression form and extracted nine quantitative features from them to train the LambdaMART sorter:

	Feature	Description
1	L2 Distance	Euclidean distance between V_{src} and V_{dest}
2	Cosine Similarity	Cosine similarity between the two vectors
3	expr_length	String length of the expression
4	sum_count	Count of $\sum$ operators
5	times_count	Count of $\times$ operators
6	neg_count	Count of $\neg$ operators
7	bracket_count	Number of [] bracket pairs
8	var_count	Number of variables (t, r, f, a)
9	complexity	Total operator count

Operator	Expression
Input _r	$f(t) := r(t)$
Proj _a	$f(t) := \sum_{t_l} (f_l(t_l) \times \llbracket t = a(t_l) \rrbracket)$
Sel _{p,a}	$f(t) := f_l(t) \times \llbracket p(a(t)) \rrbracket$
InSubSel _a	$f(t) := f_l(t) \times f_r(a(t)) \times \text{not}(\llbracket \text{IsNull}(a(t)) \rrbracket)$
IJoin _{a_l, a_r}	$f(t) := \sum_{t_l, t_r} (\llbracket t = t_l \cdot t_r \rrbracket \times f_l(t_l) \times f_r(t_r) \times \llbracket a_l(t_l) = a_r(t_r) \rrbracket \times \text{not}(\llbracket \text{IsNull}(a_l(t_l)) \rrbracket))$
LJoin _{a_l, a_r}	$f(t) := (\text{IJoin Expr}) + \sum_{t_l, t_r} (\llbracket t = t_l \cdot t_r \rrbracket \times f_l(t_l) \times \llbracket \text{IsNull}(t_r) \rrbracket \times \text{not}(\sum_{t'_r} (f_r(t'_r) \times \llbracket a_l(t_l) = a_r(t'_r) \rrbracket \times \text{not}(\llbracket \text{IsNull}(a_l(t_l)) \rrbracket))))$
RJoin _{a_l, a_r}	(symmetric to LJoin)
Dedup	$f(t) := f_l(t) $

Dual Role of LambdaMART Ranker

The LambdaMART learning-to-rank model serves **two complementary purposes** in SLER:

Phase	Role	How it Works	Key Benefit
During Enumeration (Pre-filtering)	Template-pair pre-filter	Rank all possible template pairs → select only top-k most promising pairs	Enables scalable rule discovery for complex templates ($n \geq 7$ nodes) Reduces enumeration time dramatically while retaining high- value rules
After Enumeration (Final ranking)	Rule effectiveness ranking	Score every generated rule using the 9 quantitative features → sort in descending order	Optimizes Precision@n Guarantees the top-n rules are the most effective for real-world query performance. Sort all rules.

Evaluation Setup

- **Datasets / Query Statistics:** Total workload: 11,518 unique SQL queries
 - 8,518 queries from WeTune : collected from 15 open-source applications (e.g., Discourse, Redmine)
 - 3,000 commercial queries from YashanDB : intricate relational patterns from real financial systems (kept private due to proprietary restrictions)
- **End-to-end performance evaluation:**
 - Deployed on SQL Server 2019
 - Populated with 10M+ rows (Discourse dataset)
- **Baselines**
 - WeTune (SOTA automated rule discovery)
 - No-rewrite baseline

Evaluation Setup

◆ Metrics for Evaluation:

■ Enumeration Efficiency & Scalability

- Total enumeration time and CPU hours
- Reduction in Z3 verification calls and overall verification complexity
- Scalability to complex templates ($n \geq 7$ nodes)

■ Rule Quality

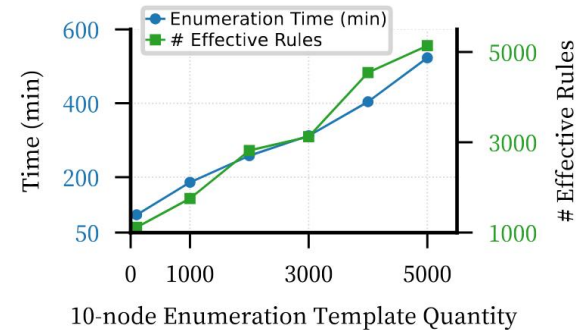
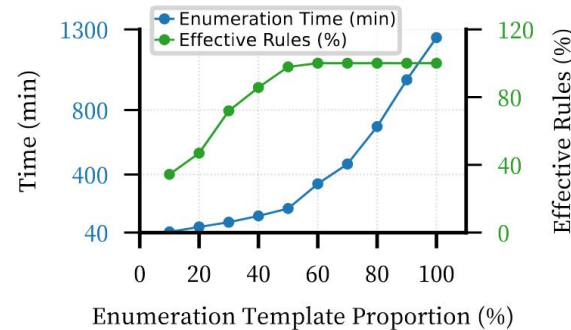
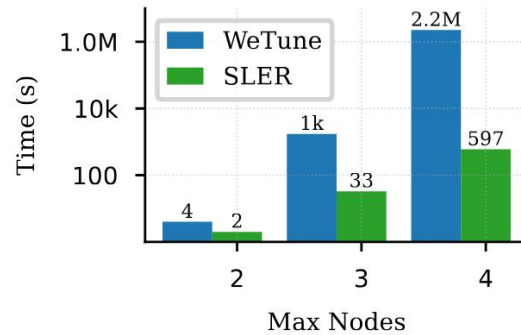
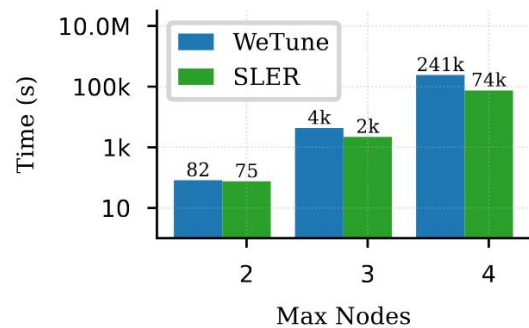
- Number of effective (non-redundant, high-value) rules discovered
- Coverage of real-world complex query patterns unreachable by prior work

■ End-to-End Optimizer Performance

- Query execution runtime on SQL Server 2019
- Plan quality (number of rewrite steps required, plan simplicity)
- Latency reduction percentage compared with WeTune and no-rewrite baseline

Main Results – Efficiency

Max nodes in the template		2	3	4	5	6
Enumeration Duration	WeTune	82s	1h 11m	67h 4m	timeout	timeout
	SLER (w.o. RTP)	72s	39m 22s	22h 59m	timeout	timeout
	SLER	75s	36m 46s	20h 49m	29d 23h	395d
De-redundancy Duration	WeTune	4s	28m 53s	20d 19h	timeout	timeout
	SLER (w.o. RTP)	3s	2m 27s	33m 33s	timeout	timeout
	SLER	2s	33s	9m 57s	10h 24m	62h 24m



Main Results – Efficiency

Max nodes in the template		2	3	4	5	6
Enumeration Duration	WeTune	82s	1h 11m	67h 4m	timeout	timeout
	SLER (w.o. RTP)	72s	39m 22s	22h 59m	timeout	timeout
	SLER	75s	36m 46s	20h 49m	29d 23h	395d
De-redundancy Duration	WeTune	4s	28m 53s	20d 19h	timeout	timeout
	SLER (w.o. RTP)	3s	2m 27s	33m 33s	timeout	timeout
	SLER	2s	33s	9m 57s	10h 24m	62h 24m



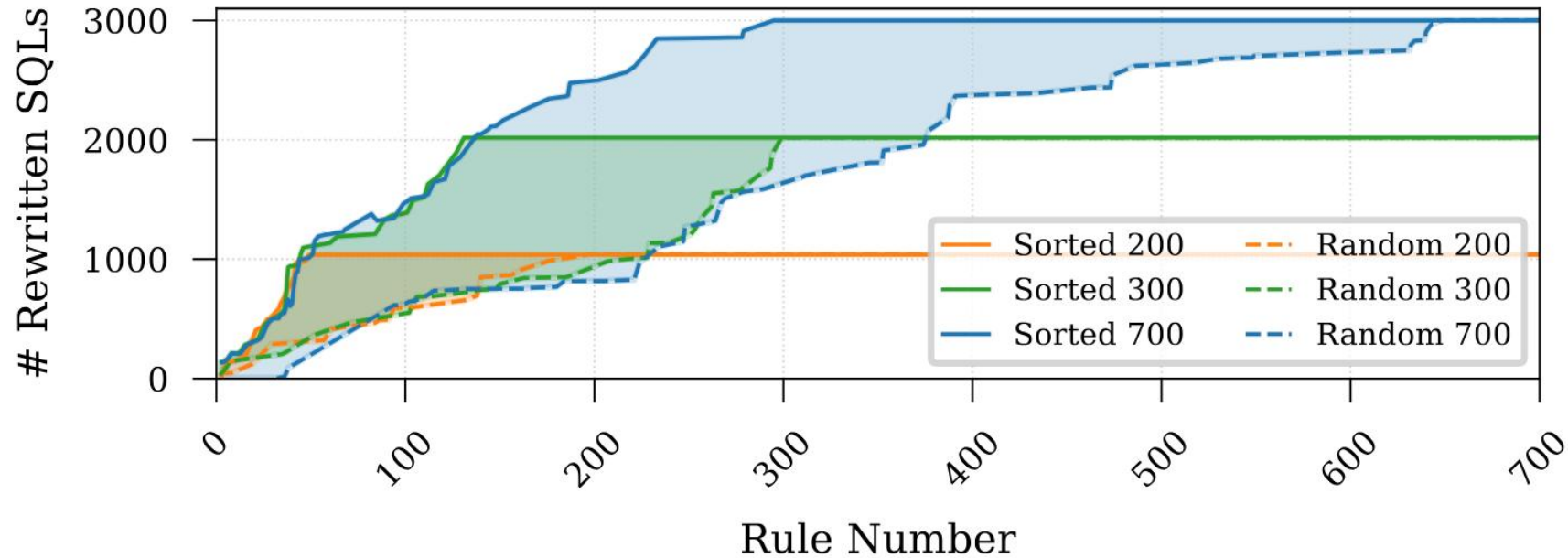
Standardized enumeration + RTP: up to **90%** reduction in verification time

Full 4-node rule discovery: dramatically **faster** than WeTune

7+ node templates now feasible via ranker pre-filtering

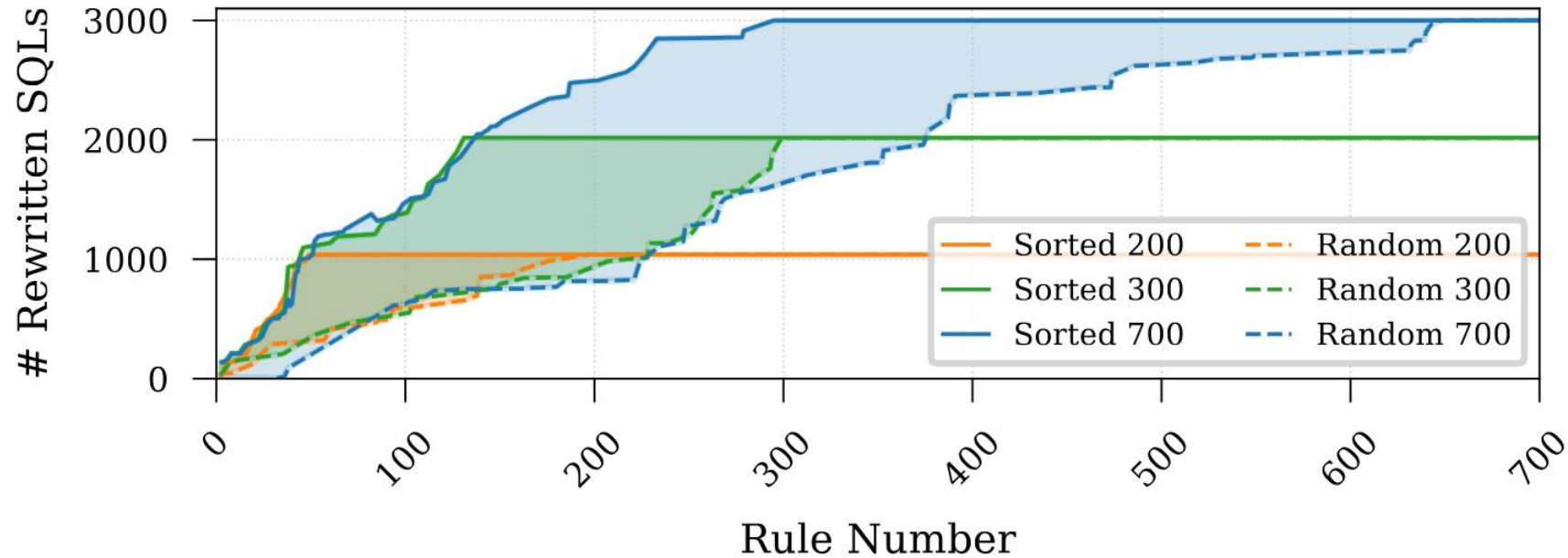
- **Example: 10-node templates** → 5,127 effective rules discovered in 523 minutes

Rule Quality



Rule Top	Rule End	Trigger Rate (%)	Top Rule Usage(%)	End Rule Usage(%)
Top 50	End 150	100	52	0
Top 500	End 200	100	34.67	0
Top 1000	End 0	100	11.4	0
Top 0	End 1000	0	0	0
Random 1000		0	0.2	0
Random 90000		0.07	13	0

Rule Quality



As the number of rules increases, the ranked rule set **consistently optimizes significantly** more queries than the random or bottom-ranked sets.

LambdaMART ranking delivers clear gains in **rewrite efficiency and query performance**.

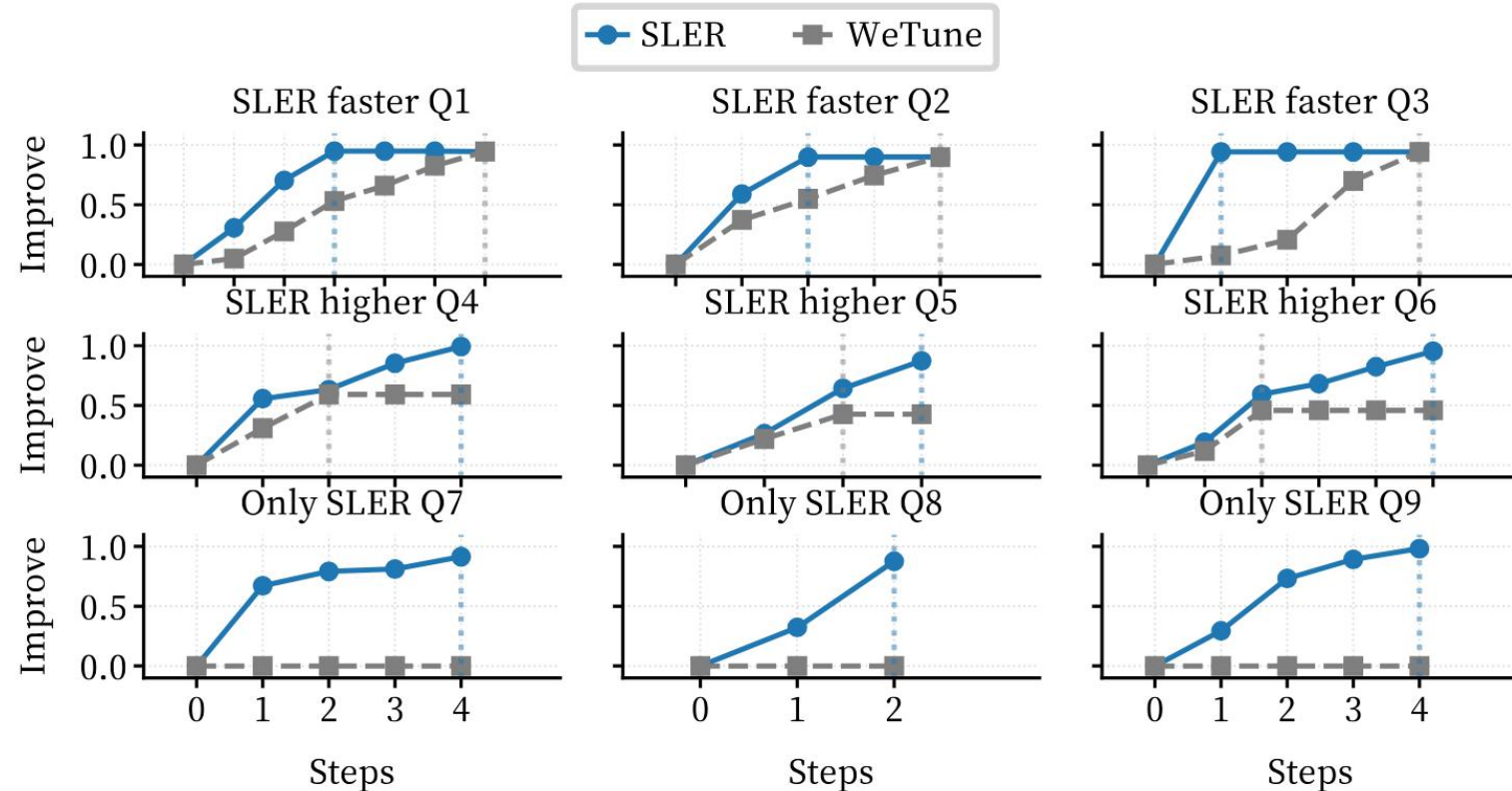
Main Results – Effectiveness

Query optimization on real workloads:

- SLER reaches ideal plans in **single step** vs. WeTune's 3–4 steps
- Latency reduction: **41.8%–73.2%** (vs. WeTune 18.5%–41.8%) in deep-optimization scenarios

Three typical win scenarios (Q1–Q9):

- convergence acceleration
- breakthrough depth
- unique optimizations



Example Rules & Impact

■ SLER discovers rules that:

- Eliminate unnecessary joins/subqueries
- Push filters/projects optimally
- Handle multi-level redundancy

No.	Source Plan Template	Destination Plan Template	Extra Constraints
1	$\text{InSub}_{a_4}(t_0, \text{Proj}_{a_1, s_1}(\text{Sel}_{p_0, a_2}(\text{InSub}_{a_1}(t_1, \text{Proj}_{a_0, s_0}(t_2))))))$	$\text{InSub}_{a_4}(t_0, \text{Proj}_{a_0, s_0}(\text{Sel}_{p_0, a_2}(\text{IJoin}_{a_1, a_0}(t_1, t_2))))$	
2	$\text{InSub}_{a_2}(\text{LJoin}_{a_0, a_0}(\text{Proj}_{a_0, s_0}^*(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_2, s_1}^*(t_0)), t_0)$	$\text{IJoin}_{a_0, a_0}(\text{Proj}_{a_0, s_0}^*(t_0), \text{Proj}_{a_2, s_1}^*(t_0))$	$\text{NotNull}(t_0, a_2)$
3	$\text{InSub}_{a_1}(\text{LJoin}_{a_1, a_0}(t_0, \text{Proj}_{a_1, s_0}^*(\text{InSub}_{a_0}(t_0, t_0))), \text{Proj}_{a_0, s_1}(t_0))$	$\text{IJoin}_{a_1, a_0}(t_0, \text{Proj}_{a_1, s_0}^*(t_0))$	$\text{NotNull}(t_0, a_0)$
4	$\text{LJoin}_{a_0, a_0}(\text{Proj}_{a_1, s_0}^*(\text{Sel}_{p_0, a_0}(t_0)), \text{Proj}_{a_0, s_1}^*(\text{Sel}_{p_0, a_0}(t_0)))$	$\text{IJoin}_{a_0, a_0}(\text{Proj}_{a_1, s_0}^*(t_0), \text{Proj}_{a_0, s_1}^*(\text{Sel}_{p_0, a_0}(t_0)))$	$\text{NotNull}(t_0, a_0)$
5	$\text{InSub}_{a_0}(\text{IJoin}_{a_0, a_1}(\text{Proj}_{a_1, s_0}^*(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_0, s_1}^*(t_0)), t_0)$	$\text{IJoin}_{a_0, a_1}(\text{Proj}_{a_0, s_0}^*(t_0), \text{Proj}_{a_1, s_1}^*(t_0))$	$\text{NotNull}(t_0, a_0)$
6	$\text{IJoin}_{a_5, a_4}(\text{Proj}_{a_1, s_1}(\text{LJoin}_{a_1, a_1}(\text{Proj}_{a_0, s_0}^*(t_0), t_0)), \text{Proj}_{a_4, s_2}^*(t_2))$	$\text{IJoin}_{a_5, a_6}(\text{Proj}_{a_1, s_1}(\text{IJoin}_{a_1, a_1}(\text{Proj}_{a_0, s_0}^*(t_0), t_0)), \text{Proj}_{a_6, s_2}^*(t_2))$	
7	$\text{Sel}_{p_0, t_0, a_4}(\text{IJoin}_{t_0, a_4, t_1, a_1}(t_0, \text{Proj}_{t_1, a_1}(\text{IJoin}_{t_1, a_1, t_2, a_0}(t_1, \text{Dedup}(\text{Proj}_{t_2, a_0}(t_2))))))$	$\text{IJoin}_{t_0, a_4, t_1, a_1}(t_0, \text{Proj}_{t_1, a_1}(\text{Sel}_{p_0, t_1, a_1}(\text{InSub}_{t_1, a_1}(t_1, \text{Proj}_{t_2, a_0}(t_2))))))$	
8	$\text{Sel}_{p_0, a_0}(\text{InSub}_{a_0}(\text{IJoin}_{a_0, a_0}(\text{Proj}_{a_0, s_0}(\text{Sel}_{p_0, a_0}(t_0)), t_0), t_0))$	$\text{LJoin}_{a_0, a_0}(\text{Proj}_{a_0, s_0}^*(\text{Sel}_{p_0, a_0}(t_0)), t_0)$	$\text{NotNull}(t_0, a_0); \text{Unique}(t_0, a_0)$
9	$\text{Proj}_{a_5, s_1}^*(\text{InSub}_{a_0}(\text{IJoin}_{a_0, a_1}(t_0, t_1), \text{Proj}_{a_2, s_0}(\text{Sel}_{p_0, a_2}(t_2))))$	$\text{Proj}_{a_5, s_1}(\text{InSub}_{a_0}(t_0, \text{Proj}_{a_2, s_0}(\text{Sel}_{p_0, a_1}(\text{LJoin}_{a_1, a_2}(t_1, t_2))))))$	$\text{Unique}(t_0, a_5)$
10	$\text{IJoin}_{a_0, a_1}(\text{Proj}_{a_1, s_0}^*(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_0, s_1}(\text{Sel}_{p_0, a_1}(t_0)))$	$\text{IJoin}_{a_0, a_1}(\text{Proj}_{a_0, s_0}^*(\text{Sel}_{p_0, a_0}(t_0)), \text{Proj}_{a_1, s_1}(\text{Sel}_{p_0, a_0}(t_0)))$	$\text{NotNull}(t_0, a_0)$
11	$\text{InSubFilter}_{a_5}(\text{InnerJoin}_{a_1, a_2}(\text{Input}_{t_0}, \text{Proj}_{a_0, s_0}^*(\text{Input}_{t_1})), \text{Proj}_{a_5, s_1}(\text{LeftJoin}_{a_1, a_2}(\text{Input}_{t_0}, \text{Input}_{t_1})))$	$\text{LeftJoin}_{a_1, a_2}(\text{Input}_{t_0}, \text{Proj}_{a_0, s_0}^*(\text{Input}_{t_1}))$	$\text{NotNull}(t_0, a_1); \text{NotNull}(t_0, a_5); \text{Reference}(t_0, a_1, t_1, a_2)$

Conclusion

- SLER solves the long-standing scalability crisis in **automated rewrite-rule discovery**.
- First system to produce **>1M high-quality rules** covering complex real-world queries.
- Template standardization + RTP + LambdaMART ranking = **general, extensible framework**.
- Opens the door to next-generation optimizers powered by massive, adaptive rule spaces.
- **Future Work**
 - Support more operators/constraints
 - Workload-specific rule subset selection
 - Integration into production DBMS

Thanks

Thank You!

Thanks

Thanks **Q & A**